

河南大学

操作系统

期末复习



张 帆

教授



13839965397



zhangfan@henu.edu.cn



计算机学院 412 房间



试卷上请填写序号（不是座号！）



目录

- 1 第二章 进程的描述与控制**
- 2 第三章 处理机调度与死锁**
- 3 第四章 存储管理**
- 4 第五章 虚拟存储器**
- 5 第六章 输入输出系统**
- 6 第七章 文件管理**
- 7 第八章 磁盘存储器的管理**



计算题类型总结

- 1 利用信号量实现进程同步
- 2 处理机调度算法（周转时间）
- 3 银行家算法
- 4 分区存储管理算法
- 5 逻辑地址物理地址变换
- 6 请求分页中的页面置换算法
- 7 磁盘访问时间
- 8 磁盘调度算法（平均移道数）
- 9 混合索引分配
- 10 文件控制块和索引结点
- 11 位示图



第 1 章

操作系统引论



进程的基本特征

结构特征 从结构上看，每个进程（进程实体）都是由程序段、数据段及进程控制块（PCB）组成。

动态性 进程的实质是程序在处理机上的一次执行过程，因此是动态性的。所以动态性是进程的最基本的特征。同时动态性还表现在进程则是有生命期的，它因创建而产生，因调度而执行，因得不到资源而暂停，因撤消而消亡。

并发性

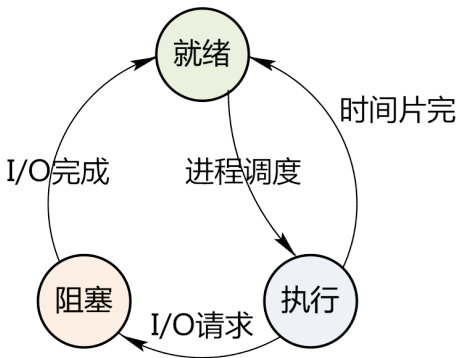
独立性

异步性

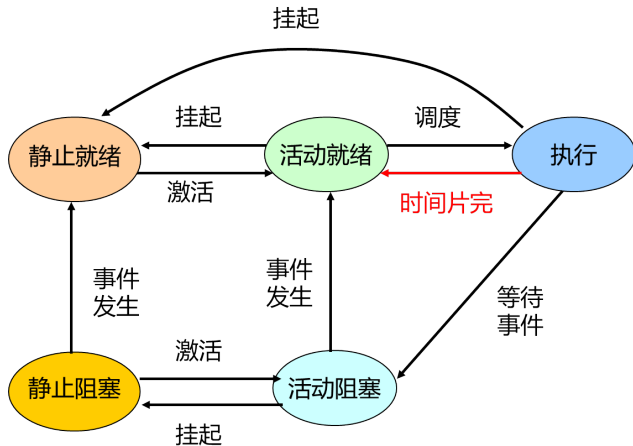


进程状态的转换

进程在运行期间并非固定处于某个状态，而是不断从一个状态转换到另一个状态。



具有挂起状态的进程状态转换



线程的基本概念

- 进程是资源分配的单位，而线程是处理机调度的单位。
- 一个进程可以创建一个或多个线程。
- 多个线程会争夺 CPU，在不同的状态之间进行转换。
线程也是一个动态的概念，也有一个从创建到消亡的生命过程，具多种状态。
- 同一进程中的所有线程都具有相同的地址空间（进程的地址空间）。



线程的实现方式

1 内核支持线程的实现

直接利用系统调用进行线程控制。

2 用户级线程的实现

不能直接利用系统调用。为了取得内核的服务（获得系统资源），需要借助中间系统（运行时系统、轻型进程）。



同步机制应遵循的规则

- 1 空闲让进**：当无进程处于临界区时，表明临界资源处于空闲状态，应允许一个请求进入临界区的进程立即进入自己的临界区，以有效地利用临界资源。
- 2 忙则等待**：当已有进程进入临界区时，表明临界资源正在被访问，因而其他试图进入临界区的进程必须等待，以保证对临界资源的互斥访问。
- 3 有限等待**：对要求访问临界资源的进程，应保证在有限时间内能进入自己的临界区，以免陷入 **死等** 状态。
- 4 让权等待**：当进程不能进入自己的临界区时，应立即释放处理机，以免进程陷入 **忙等**。



信号量机制

信号量: 用于表示资源数目或请求使用某一资源的进程个数的整型量。

- 整型信号量
- 记录型信号量
- 信号量集 (AND 信号量集、一般信号量集)

信号量只能通过初始化和两个标准的原语 (P, V 操作) 来访问。



利用信号量实现进程互斥

例：进程 A 与进程 B 共享同一文件 file，设此文件要求互斥使用，则可将 file 作为临界资源，有关 file 的使用程序段分别为临界区 CSA 和 CSB。

解：semaphore mutex=1;

PA:

L1: P(mutex);

CSA;

V(mutex);

remainder of process A;

GOTO L1;

PB:

L2: P(mutex);

CSB;

V(mutex);

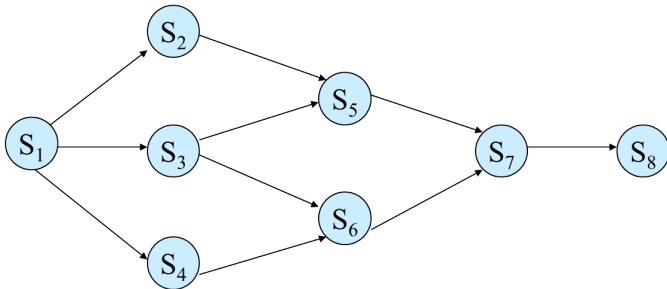
remainder of process B;

GOTO L2;



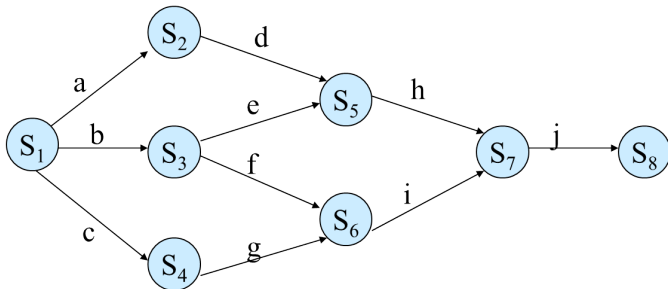
利用信号量实现进程同步

例：利用信号量来描述前趋图关系

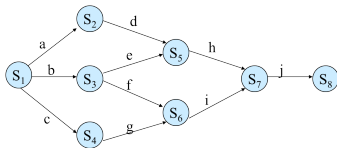


利用信号量实现进程同步

这是一个具有 8 个结点的前趋图。图中的前趋图中共有 10 条有向边，可设 10 个信号量，初值均为 0；有 8 个结点，可设计成 8 个并发进程，具体描述如下：



利用信号量实现进程同步



```

1  Struct smaphore a,b,c,d,e,f,g,h,i,j=0,0,0,0,0,0,0,0,0,0
2  cobegin
3      {S1;V(a);V(b);V(c);}
4      {P(a);S2;V(d);}
5      {P(b);S3;V(e);V(f);}
6      {P(c);S4;V(g);}
7      {P(d);P(e);S5;V(h);}
8      {P(f);P(g);S6;V(i);}
9      {P(h);P(i);S7;V(j);}
10     {P(j);S8;}
11  coend
  
```



经典进程同步问题

在多道程序环境下，进程同步问题十分重要，出现一系列经典的进程同步问题，其中有代表性有：

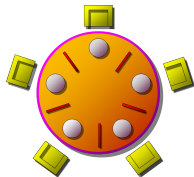
经典进程同步问题

- 生产者—消费者问题
- 哲学家进餐问题
- 读者—写者问题



“哲学家进餐”问题

- 有五个哲学家，他们的生活方式是交替地进行思考和进餐。他们共用一张圆桌，分别坐在五张椅子上。
- 在圆桌上有五个碗和五支筷子，平时一个哲学家进行思考，饥饿时便试图取用其左、右最靠近他的筷子，只有在他拿到两支筷子时才能进餐。进餐完毕，放下筷子又继续思考。



哲学家进餐问题可看作是并发进程并发执行时处理共享资源的一个有代表性的问题。



“哲学家进餐”问题

```
Var chopstick: array[0 , ... , 4] of semaphore=1 ;  
/*5支筷子分别设置为初始值为1的互斥信号量*/
```

第*i*个哲学家的活动

```
repeat  
    wait(chopstick[i]) ;  
    wait(chopstick[(i+1) mod 5]) ;  
    eat ;  
    signal(chopstick[i]) ;  
    signal(chopstick[(i+1) mod 5]) ;  
    think ;  
until false ;
```



“哲学家进餐”问题

- 此算法可以保证不会有相邻的两位哲学家同时进餐。
- 若五位哲学家同时饥饿而各自拿起了左边的筷子，这使五个信号量 chopstick 均为 0，当他们试图去拿起右边的筷子时，都将因无筷子而无限期地等待下去，即可能会引起死锁。



哲学家进餐问题的改进解法

- 方法一：至多只允许四位哲学家同时去拿左筷子，最终能保证至少有一位哲学家能进餐，并在用完后释放两只筷子供他人使用。
- 方法二：仅当哲学家的左右手筷子都拿起时才允许进餐。
- 方法三：规定奇数号哲学家先拿左筷子再拿右筷子，而偶数号哲学家相反。



哲学家进餐问题的改进解法

- 方法一：至多只允许四位哲学家同时去拿左筷子，最终能保证至少有一位哲学家能进餐，并在用完后释放两只筷子供他人使用。
 - 设置一个初值为 4 的信号量 r ，只允许 4 个哲学家同时去拿左筷子，这样就能保证至少有一个哲学家可以就餐，不会出现饿死和死锁的现象。
 - 原理：至多只允许四个哲学家同时进餐，以保证至少有一个哲学家能够进餐，最终总会释放出他所使用过的两支筷子，从而可使更多的哲学家进餐。



哲学家进餐问题的改进解法

```
semaphore chopstick[5]={1 , 1 , 1 , 1 , 1};
semaphore r=4;
void philosopher(int i)
{
    while(true)
    {
        think();
        wait(r);                                //请求进餐
        wait(chopstick[i]);                     //请求左手边的筷子
        wait(chopstick[(i+1) mod 5]);           //请求右手边的筷子
        eat();
        signal(chopstick[(i+1) mod 5]);         //释放右手边的筷子
        signal(chopstick[i]);                   //释放左手边的筷子
        signal(r);                              //释放信号量r
        think();
    }
}
```



哲学家进餐问题的改进解法

- 方法二：仅当哲学家的左右手筷子都拿起时才允许进餐。
 - 解法 1：利用 AND 型信号量机制实现。
 - 原理：多个临界资源，要么全部分配，要么一个都不分配，因此不会出现死锁的情形。



哲学家进餐问题的改进解法

```
semaphore chopstick[5]={1 , 1 , 1 , 1 , 1};  
void philosopher(int i)  
{  
    while(true)  
    {  
        think();  
        Swait(chopstick[i]; chopstick[(i+1) mod 5]);  
        eat();  
        Ssignal(chopstick[(i+1) mod 5]; chopstick[i]);  
        think();  
    }  
}
```



哲学家进餐问题的改进解法

- 方法二：仅当哲学家的左右手筷子都拿起时才允许进餐。
 - 解法 2：利用信号量的保护机制实现。
 - 原理：通过互斥信号量 mutex 对 eat() 之前取左侧和右侧筷子的操作进行保护，可以防止死锁的出现。



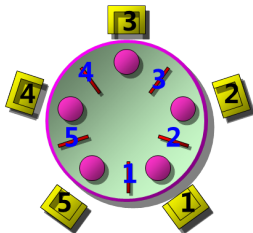
哲学家进餐问题的改进解法

```
semaphore mutex = 1;
semaphore chopstick[5]={1 , 1 , 1 , 1 , 1};
void philosopher(int i)
{
    while(true)
    {
        think();
        wait(mutex);
        wait(chopstick[i]);
        wait(chopstick [(i+1) mod 5]);
        signal(mutex);
        eat();
        signal(chopstick [(i+1) mod 5]);
        signal(chopstick[i]);
        think();
    }
}
```



哲学家进餐问题的改进解法

- 方法三：规定奇数号哲学家先拿左筷子再拿右筷子，而偶数号哲学家相反。
- 原理：按照下图，将是 2,3 号哲学家竞争 3 号筷子，4,5 号哲学家竞争 5 号筷子。1 号哲学家不需要竞争。最后总会有一个哲学家能获得两支筷子而进餐。



哲学家进餐问题的改进解法

```

semaphore chopstick[5]={1, 1, 1, 1, 1};
void philosopher(int i)
{
    while(true)
    {
        if(i mod 2 == 0)                                //偶数哲学家，先右后左。
        {
            wait (chopstick [(i + 1) mod 5]);
            wait (chopstick [i]);
            eat();
            signal (chopstick [i]);
            signal (chopstick[(i+1) mod 5]);
        }
        Else                                            //奇数哲学家，先左后右。
        {
            wait (chopstick [i]);
            wait (chopstick [(i+1) mod 5]);
            eat();
            signal (chopstick [(i+1) mod 5]);
            signal (chopstick [i]);
        }
    }
}

```

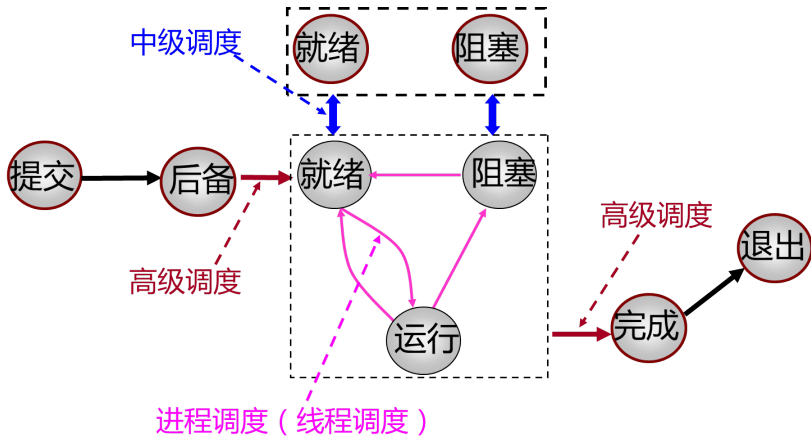


第 3 章

处理机调度与死锁



处理机调度的层次



处理机调度的层次



先来先服务调度算法 FCFS

- 基本思想：按进程（作业）进入就绪（后备）队列的先后次序来分配处理机（为其创建进程）。
- 一般采用非剥夺的调度方式。

例：

进程名	到达时间	服务时间
A	0	1
B	1	100
C	2	1
D	3	100



上一个进程的
完成时间

开始时间+
服务时间

完成时间-
到达时间

周转时间
服务时间

进程名	到达时间	服务时间	开始执行时间	完成时间	周转时间	带权周转时间
A	0	1	0	1	1	1
B	1	100	1	101	100	1
C	2	1	101	102	100	100
D	3	100	102	202	199	1.99



短作业优先调度算法 (非抢占)

作业	到达时间 T_{in}	服务时间 T_r	开始时间 T_s	结束时间 T_c	周转时间 T	带权周转时间 W
A	0	3				
B	2	6				
C	4	4				
D	6	5				
E	8	2				
平均时间						

执行顺序：

周转时间 = 结束时间 - 到达时间
带权周转时间 = 周转/服务



短作业优先调度算法 (非抢占)

作业	到达时间 T_{in}	服务时间 T_r	开始时间 T_s	结束时间 T_c	周转时间 T	带权周转时间 W
A	0	3	0	3	3	1
B	2	6	3	9	7	1.17
C	4	4	11	15	11	2.75
D	6	5	15	20	14	2.8
E	8	2	9	11	3	1.5
平均时间					7.6	1.84

执行顺序：A → B → E → C → D

周转时间 = 结束时间 - 到达时间
带权周转时间 = 周转 / 服务



高响应比优先调度算法

$$\text{优先权} = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}}$$

- 如等待时间相同，则要求服务时间越短其优先权越高
→SJF。
- 如要求服务时间相同，优先权决定于等待时间
→FCFS。
- 对长作业，若等待时间足够长，优先权也高，也能获得CPU。



高响应比优先调度算法

进程	到达时间	服务时间	完成时间	周转时间	带权周转时间
A	0	20	20	20	1
B	5	15			
C	10	5			
D	15	10			

.1

□



高响应比优先调度算法

进程	到达时间	服务时间	完成时间	周转时间	带权周转时间
A	0	20	20	20	1
B	5	15	40	35	2.33
C	10	5	25	15	3
D	15	10	50	35	3.5

20时刻: $R_b = (15+15)/15=2$ $R_c = (10+5)/5=3$ $R_d = (5+10)/10=1.5$

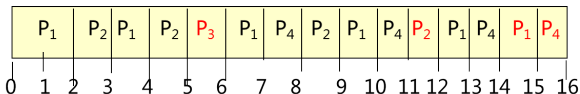
25时刻: $R_b = (20+15)/15=2.33$ $R_d = (10+10)/10=2$



时间片轮转调度算法 (Round Robin, RR)

进程	到达时间	服务时间
P1	0	7
P2	2	4
P3	4	1
P4	5	4

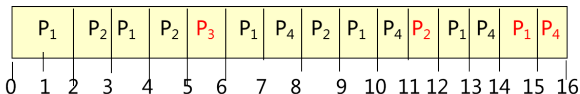
RR (时间片为 1)



时间片轮转调度算法 (Round Robin, RR)

进程	到达时间	服务时间
P1	0	7
P2	2	4
P3	4	1
P4	5	4

RR (时间片为 1)



- 平均周转时间 $= ((15-0) + (12-2) + (6-4) + (16-5)) / 4 = 9.5$
- 平均等待时间 $= (8 + 6 + 1 + 7) / 4 = 5.5$



多级反馈队列调度算法 MFQ

■ 多级反馈队列调度算法

是时间片轮转算法和优先级调度算法的综合和发展，通过动态调整进程优先级和时间片大小，不必事先估计进程的执行时间。

■ FCFS + 优先级 + RR + 抢占

■ 多级反馈队列可兼顾多方面的系统目标，是目前公认的一种较好的进程调度算法。



多级反馈队列调度算法调度机制

- 1 设置多个就绪队列，并为每个队列赋予不同的优先级。队列 1 的优先级最高，其余队列逐个降低。
- 2 每个队列中进程执行时间片的大小各不相同，进程所在队列的优先级越高，其相应的时间片就越短。
- 3 新进程进入系统时，先放入队列 1 的末尾，按FCFS等待调度。如能完成，便可准备撤离系统，反之由调度程序将其转入队列 2 的末尾，按 FCFS 再次等待调度，如此下去，最后进入队列 n 按RR算法调度执行。
- 4 仅当队列 1 为空时，才调度队列 2 中的进程运行。若一个队列中的进程正执行，此时有新进程进入高级队列，则新进程抢占运行，原进程转移至本队列队尾。



死锁

■ 产生死锁的原因

- 1 竞争资源
- 2 进程间推进顺序非法

■ 产生死锁的四个必要条件

- 1 互斥条件
- 2 请求和保持条件
- 3 不可剥夺条件
- 4 环路等待条件



处理死锁的方法

- 1 **预防死锁**: 通过设置某些限制条件, 去破坏产生死锁的四个必要条件中的一个或几个条件, 来防止死锁的发生。
- 2 **避免死锁**: 在资源的动态分配过程中, 用某种方法去防止系统进入不安全状态, 从而避免死锁的发生。
- 3 **检测死锁**: 允许系统在运行过程中发生死锁, 但可设置检测机构及时检测死锁的发生, 并采取适当措施加以清除。
- 4 **解除死锁**: 当检测出死锁后, 便采取适当措施将进程从死锁状态中解脱出来。



银行家算法练习题

例：T0 时刻的资源分配情况如下：

	最大资源需求			已分配资源数量		
	A	B	C	A	B	C
P1	5	5	9	2	1	2
P2	5	3	6	4	0	2
P3	4	0	11	4	0	5
P4	4	0	5	2	0	4
P5	4	2	4	3	1	4

剩余资源向量为：Available = (2, 3, 3)

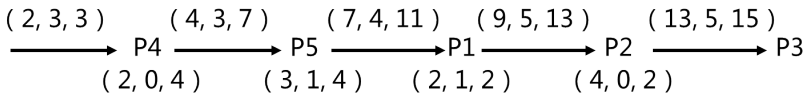
- 1 T0 时刻是否为安全状态？若是，请给出安全序列。
- 2 在 T0 时刻若进程 P2 请求资源 (0, 3, 4)，是否能实施资源分配？
- 3 若进程 P4 请求资源 (2, 0, 0)，是否可以资源分配？



银行家算法练习题 剩余向量 = (2, 3, 3)

	最大资源需求Max			已分配资源数量			尚需资源Need		
	A	B	C	A	B	C	A	B	C
P1	5	5	9	2	1	2	3	4	7
P2	5	3	6	4	0	2	1	3	4
P3	4	0	11	4	0	5	0	0	6
P4	4	0	5	2	0	4	2	0	1
P5	4	2	4	3	1	4	1	1	0

1 T0 时刻是否为安全状态？若是，请给出安全序列。



T0 时刻为安全状态，安全序列：p4, p5, p1, p2, p3



银行家算法练习题 剩余向量 = (2, 3, 3)

	最大资源需求Max			已分配资源数量			尚需资源Need		
	A	B	C	A	B	C	A	B	C
P1	5	5	9	2	1	2	3	4	7
P2	5	3	6	4	0	2	1	3	4
P3	4	0	11	4	0	5	0	0	6
P4	4	0	5	2	0	4	2	0	1
P5	4	2	4	3	1	4	1	1	0

- 2 在 T0 时刻若进程 P2 请求资源 (0, 3, 4)，是否能实施资源分配？

因为 Request₂ (0, 3, 4) > Available (2, 3, 3)
所以不能满足进程 P2 的请求。



银行家算法练习题

	最大资源需求Max			已分配资源数量			尚需资源Need		
	A	B	C	A	B	C	A	B	C
P1	5	5	9	2	1	2	3	4	7
P2	5	3	6	4	0	2	1	3	4
P3	4	0	11	4	0	5	0	0	6
P4	4	0	5	2	0	4	2	0	1
P5	4	2	4	3	1	4	1	1	0

3 若进程 P4 请求资源 (2, 0, 0), 是否可以资源分配?

$$\text{Request}_4 (2, 0, 0) \leq \text{Need}_4 (2, 0, 1)$$

$$\text{且 } \text{Request}_4 (2, 0, 0) < \text{Available} (2, 3, 3)$$

试分配后, 则 $\text{Need}_4 = (0, 0, 1)$, $\text{Allocation}_4 = (4, 0, 4)$

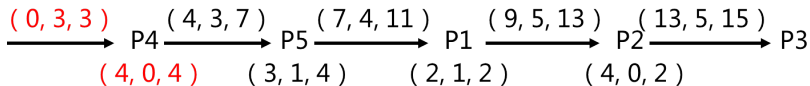
新的剩余向量 $\text{Available} = (0, 3, 3)$



银行家算法练习题

	最大资源需求 Max			已分配资源数量			尚需资源Need		
	A	B	C	A	B	C	A	B	C
P1	5	5	9	2	1	2	3	4	7
P2	5	3	6	4	0	2	1	3	4
P3	4	0	11	4	0	5	0	0	6
P4	4	0	5	4	0	4	0	0	1
P5	4	2	4	3	1	4	1	1	0

新的剩余向量 Available = (0, 3, 3)



系统安全，安全序列：p4, p5, p1, p2, p3，可以分配



第 4 章

存储管理



连续分配方式（分区技术）

- 单一连续分区分配（静态分区技术）：仅用于单用户单任务系统
- 固定分区分配（静态分区技术）：可用于多道系统
- 动态分区分配（动态分区技术）
- 动态可重定位分区分配（动态分区技术）：引入了动态重定位和内存紧凑技术
- 伙伴系统（动态分区技术）



动态分区分配算法

1 基于顺序搜索的动态分区分配算法

- 首次适应算法 (First Fit)
- 循环首次适应算法 (Next Fit)
- 最佳适应算法 (Best Fit)
- 最坏适应算法 (Worst Fit)

2 基于索引搜索的动态分区分配算法

- 快速适应算法 (Quick Fit)
- 伙伴系统 (Buddy system)
- 哈希算法



离散分配方式

- 连续分配存储管理方式产生的问题：
①要求连续的存储区 ②碎片问题
- 变连续分配为离散分配，允许将作业离散放到多个不相邻接的分区中。

离散分配方式

- 1 分页式存储管理：离散分配的基本单位是页
- 2 分段式存储管理：离散分配的基本单位是段
- 3 段页式存储管理：离散分配的基本单位是段、页



页面和物理块

■ 空间划分

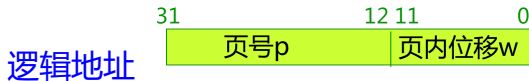
- 1 将一个用户进程的地址空间（逻辑空间）划分成若干个大小相等的区域，称为**页**或页面，各页从 0 开始编号。
- 2 内存空间也分成若干个**与页大小相等**的区域，称为**块**（物理块）或页框（frame），同样从 0 开始编号。

■ 内存分配

- 在为进程分配内存时以块为单位，将进程中若干页装入到多个不相邻的块中，最后一页常装不满一块而出现**页内碎片**。

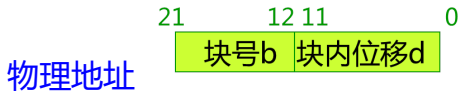


地址结构



例：地址长为 32 位，其中 0-11 位为页内地址，即每页的大小为 $2^{12}=4\text{KB}$ ；

12-31 位为页号，地址空间最多允许有 $2^{20}=1\text{M}$ 页。



例：地址长为 22 位，其中 0-11 位为块内地址，即每块的大小为 $2^{12}=4\text{KB}$ ，与页相等；

12-21 位为块号，内存地址空间最多允许有 $2^{10}=1\text{K}$ 块。



地址变换例题

例：若在一分页存储管理系统中，某作业的页表如下表所示，已知页面大小为 1024B，试将十进制逻辑地址 1011 转化为相应的物理地址。

页号	块号
0	2
1	3
2	1
3	6



地址变换例题

设页号为 P ，页内位移为 W ，逻辑地址为 A ，内存地址为 M ，页面大小为 L ，则

$$P = \text{int} (A / L)$$

$$W = A \bmod L$$

对于逻辑地址 1011

$$P = \text{int}(1011/1024) = 0$$

$$W = 1011 \bmod 1024 = 1011$$

$$A = 1011 = (0, 1011)$$

查页表第 0 页在第 2 块，所以物理地址为 $M = 1024 * 2 + 1011 = 3059$ 。

页号	块号
0	2
1	3
2	1
3	6



访问内存的有效时间

- **有效访问时间**(Effective Access Time ,EAT) 是指从给定逻辑地址, 经过地址变换, 到在内存中找到对应物理地址单元并取出数据所用的总时间。
- **基本地址变换机构**
设 T_M 为内存的访问时间

$$EAT = 2T_M$$

- **具有快表的地址变换机构**
设 P_{TLB} 为快表的命中率, T_{TLB} 为快表的访问时间

$$EAT = P_{TLB} * (T_{TLB} + T_M) + (1 - P_{TLB}) * (T_{TLB} + 2T_M)$$



访问内存的有效时间

例：有一页式系统，其页表存放在内存中。（1）如果对内存的一次存取需要 100ns，试问实现一次页面访问的存取时间是多少？（2）如果有快表，对快表的一次存取需要 20ns，平均命中率为 85%，试问此时的存取时间为多少？

- 1 页表放内存中，则实现一次页面访问需 2 次访问内存。
所以实现一次页面访问的存取时间为： $100\text{ns} \times 2 = 200\text{ns}$
- 2 系统有快表，则实现一次页面访问的存取时间为：
 $0.85 \times (20\text{ns} + 100\text{ns}) + (1 - 0.85) \times (20\text{ns} + 2 \times 100\text{ns}) = 135\text{ns}$



分段系统

- 在为作业分配内存时以段为单位，分配一段连续的物理地址空间，段间不必连续。
- 分页管理中，作业地址空间是一维的，逻辑地址是的线性地址。
- 分段管理中，整个作业的地址空间由于是分成多个段，因而是二维的，其逻辑地址由段号和段内地址所组成。



分段地址变换例题

例：在一个段式存储管理系统中，其段表为：

段号	内存起始地址	段长
0	210	500
1	2350	20
2	100	90
3	1350	590
4	1938	95

0	430
2	120

试求右表中逻辑地址对应的物理地址是什么？

解：逻辑地址

0	430
---	-----

 对应的物理地址为： $210+430=640$ 。

逻辑地址

2	120
---	-----

 段内地址 $120 >$ 段长 90 ，所以该段为非法段。



第 5 章

虚拟存储器



存储管理分类

■ 实存管理

- 分区 (Partitioning) (连续分配方式) (包括固定分区、可变分区)
- 分页 (Paging)
- 分段 (Segmentation)
- 段页式 (Segmentation with paging)



存储管理分类

■ 实存管理

- 分区 (Partitioning) (连续分配方式) (包括固定分区、可变分区)
- 分页 (Paging)
- 分段 (Segmentation)
- 段页式 (Segmentation with paging)

■ 虚存管理

- 请求分页 (Demand paging) -- 主流技术
- 请求分段 (Demand segmentation)
- 请求段页式 (Demand SWP)



存储管理分类

■ 实存管理

- 分区 (Partitioning) (连续分配方式) (包括固定分区、可变分区)
- 分页 (Paging)
- 分段 (Segmentation)
- 段页式 (Segmentation with paging)

■ 虚存管理

- 请求分页 (Demand paging) -- 主流技术
- 请求分段 (Demand segmentation)
- 请求段页式 (Demand SWP)

离散分配



虚拟存储技术的概念

- 速度和容量：虚拟存储量的扩大是以牺牲 CPU 工作时间以及内外存交换时间为代价。
- 虚拟存储器的容量取决于主存与辅存的容量，最大容量是由计算机的地址结构决定。
- 虚拟存储器的逻辑地址空间理论上不受物理存储器的限制。
如 32 位机器，虚拟存储器的最大容量就是 4G，再大 CPU 无法直接访问。
- 虚拟存储器的实现方法：请求分页、请求分段、请求段页式



请求分页系统

- 在分页系统的基础上，增加了请求调页功能、页面置换功能所形成的页式虚拟存储器系统。
- 它允许只装入若干页的用户程序和数据，便可启动运行，以后在硬件支持下通过调页功能和置换页功能，陆续将要运行的页面调入内存，同时把暂不运行的页面换到外存上，置换时以页面为单位。
- 系统须设置相应的硬件支持和软件：
 - 1 硬件支持：请求分页的页表机制、缺页中断机构和地址变换机构。
 - 2 软件：请求调页功能和页置换功能的软件。



页面置换算法

- 最佳置换算法 OPT：选择永远不再需要的页面或最长时间以后才需要访问的页面予以淘汰。
- 先进先出置换算法 FIFO：选择先进入内存的页面予以淘汰。
- 最近最久未使用置换算法 LRU：选择最近一段时间最长时间没有被访问过的页面予以淘汰。



最佳置换算法 OPT

例：假定系统为某进程分配了 3 个物理块，进程运行时的页面走向为 1,2,3,4,1,2,5,1,2,3,4,5，开始时 3 个物理块均为空，计算采用最佳置换算法时的缺页率？

页面走向	1	2	3	4	1	2	5	1	2	3	4	5
物理块1	1	1	1	1	1	1	1	1	1	3	3	3
物理块2		2	2	2	2	2	2	2	2	2	4	4
物理块3			3	4	4	4	5	5	5	5	5	5
缺页	缺	缺	缺	缺			缺			缺	缺	



最佳置换算法 OPT

例：假定系统为某进程分配了 3 个物理块，进程运行时的页面走向为 1,2,3,4,1,2,5,1,2,3,4,5，开始时 3 个物理块均为空，计算采用最佳置换算法时的缺页率？

页面走向	1	2	3	4	1	2	5	1	2	3	4	5
物理块1	1	1	1	1	1	1	1	1	1	3	3	3
物理块2		2	2	2	2	2	2	2	2	2	4	4
物理块3			3	4	4	4	5	5	5	5	5	5
缺页	缺	缺	缺	缺			缺			缺	缺	

缺页率 7/12



先进先出置换算法 FIFO

先进先出置换算法 FIFO：选择先进入内存的页面予以淘汰。

例：假定系统为某进程分配了**3 个物理块**，进程运行时的页面走向为 1,2,3,4,1,2,5,1,2,3,4,5，开始时 3 个物理块均为空，计算采用**先进先出置换算法**时的缺页率？

页面走向	1	2	3	4	1	2	5	1	2	3	4	5
物理块1	1	1	1	4	4	4	5	5	5	5	5	5
物理块2		2	2	2	1	1	1	1	1	3	3	3
物理块3			3	3	3	2	2	2	2	2	4	4
缺页	缺	缺	缺	缺	缺	缺	缺			缺	缺	

缺页率 9/12



最近最久未使用算法 (LRU)

LRU：淘汰最近一段时间最长时间没有被访问过的页面。

例：假定系统为某进程分配了**3 个物理块**，进程运行时的页面走向为 1,2,3,4,1,2,5,1,2,3,4,5，开始时 3 个物理块均为空，计算采用**最近最久未使用算法**时的缺页率？

页面走向	1	2	3	4	1	2	5	1	2	3	4	5
物理块1	1	1	1	4	4	4	5	5	5	3	3	3
物理块2		2	2	2	1	1	1	1	1	1	4	4
物理块3			3	3	3	2	2	2	2	2	2	5
缺页	缺	缺	缺	缺	缺	缺	缺			缺	缺	缺

缺页率 10/12



第 6 章

输入输出系统



假脱机技术

■ 脱机输入输出技术

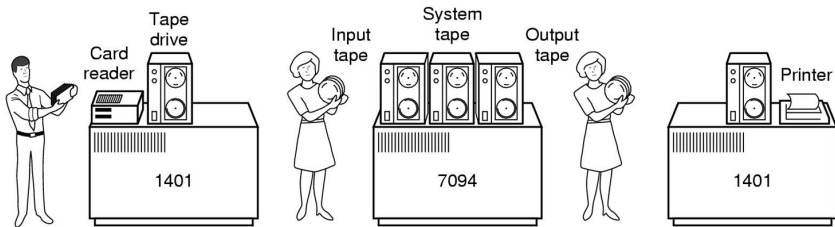
为了缓和 CPU 的高速性与 I/O 设备的低速性间矛盾而引入，该技术在外围控制机的控制下实现低速的 I/O 设备与高速的磁盘之间进行数据传送。

- **SPOOLing** (Simultaneous Peripheral Operations On-Line, 外部设备联机并行操作), 也称**假脱机技术**。
- SPOOLing 是指在多道程序的环境下, 利用多道程序中的一道或两道程序来模拟外围控制机, 从而在联机的条件下实现脱机 I/O 的功能。



假脱机技术

- 设计思想：用常驻内存的进程去外围机，从而用一台主机完成脱机技术中需用三台计算机完成的工作。
- 功能：
 - 1 把独占设备改造为逻辑共享设备。
 - 2 把一台物理 I/O 设备虚拟为多台逻辑 I/O 设备。



SPOOLing 系统的组成

SPOOLing 系统的组成

- 1 输入井和输出井
- 2 输入缓冲区和输出缓冲区
- 3 输入进程和输出进程
- 4 井管理程序



磁盘性能简述

■ 数据的组织

- 磁盘结构：磁道、柱面、扇区
- 磁盘物理块的地址：磁头号、柱面号、扇区号
- 存储容量 = 磁头（盘面）数 × 磁道（柱面）数 × 每道扇区数 × 每扇区字节数

- 假定一块硬盘磁头数为 4，柱面数为 2048，每个磁道有扇区 2048，每个扇区记录 1KB（字节），该硬盘储存容量为：

$$4 \times 2048 \times 2048 \times 1024 / 1024 / 1024 / 1024 = 16 \text{ G}$$

$$4 \times 2048 \times 2048 \times 1024 / 1000 / 1000 / 1000 = 17.2 \text{ G} \quad (\text{厂家})$$



磁盘访问时间

- **寻道时间 T_s** : 将磁头从当前位置移到指定磁道所经历的时间 $T_s = m \times n + s$ 。s 为启动磁臂的时间, n 为移动的磁道数, m 为常数。
- **旋转延迟时间 T_r** : 指定扇区移动到磁头下面所经历的时间。设每秒 r 转, 则 $T_r = 1/2r$ (均值)
- **传输时间 T_t** : 将扇区上的数据从磁盘读出或向磁盘写入数据所经历的时间 $T_t = b/rN$ 。其中 b: 读写字节数, N: 每道上的字节数。
- **控制器时间 T_c**

$$\text{访问时间 } T_a: T_a = m \times n + s + 1/2r + b/rN + T_c$$



磁盘访问时间

例：假设磁盘空闲（没有排队延迟）。平均寻道时间是 9ms，传输速度是每秒 4MB，转速是 7200rpm，控制器的开销是 1ms。问读或写一个 512 字节的扇区的平均时间是多少？

访问时间=**寻道时间** + **旋转延迟** + **传输时间** + **控制器时间**

1 秒等于 1000 毫秒

解：**访问时间**= $9\text{ms} + 0.5/7200(\text{rpm}) + 0.5(\text{k})/4(\text{MB/s}) + 1\text{ms}$
 $= 9\text{ms} + 0.5/7200/60/1000 + 0.5/(4 \times 1024/1000) + 1\text{ms}$
 $\approx 9 + 4.2 + 0.122 + 1 = 14.322\text{ms}$



磁盘调度算法

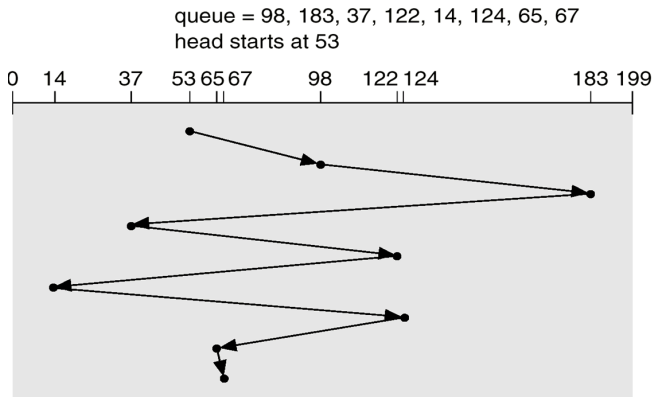
- 早期的磁盘调度算法
 - 先来先服务 FCFS
 - 最短寻道时间优先 SSTF
- 扫描算法
 - 扫描 (SCAN)/电梯 (LOOK) 算法
 - 循环扫描 (CSCAN) 算法
 - N-STEP-SCAN 调度算法和 FSCAN 调度算法

本教材不区分 SCAN 算法和 LOOK 算法，请大家都按
LOOK 算法解决

不扫描到头！



先来先服务调度算法

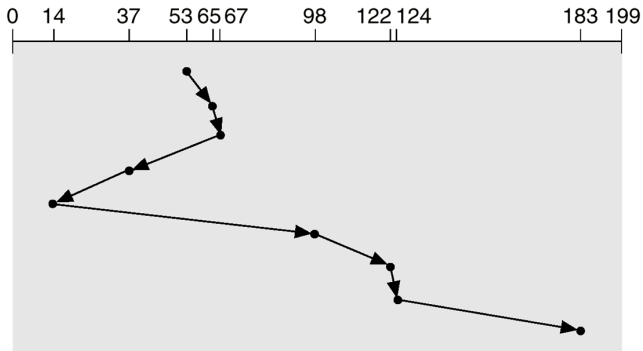


下磁道	移道数
98	45
183	85
37	146
122	85
14	108
124	110
65	59
67	2
总道数	640
平均	80



最短寻道时间优先算法

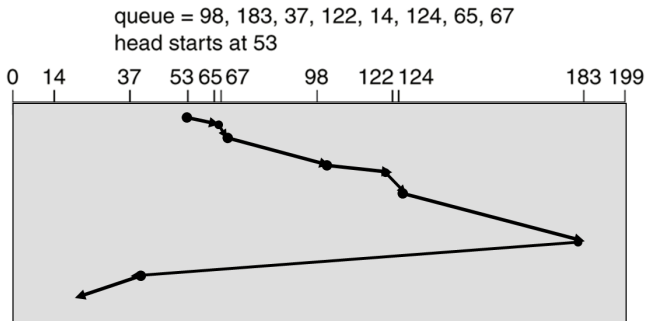
queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53



下磁道	移道数
65	12
67	2
37	30
14	23
98	84
122	24
124	2
183	59
总道数	236
平均	29.5



电梯算法 (Look 算法)



下磁道	移道数
65	12
67	2
98	31
122	24
124	2
183	59
37	146
14	23
总道数	299
平均	37.4



第 7 章

文件管理



文件的逻辑结构

所有文件存在着两种形式的结构：

1 文件的逻辑结构（文件组织）

从用户观点出发所观察到的文件组织形式，是用户可以
直接处理的数据及其结构，它独立于物理特性。

顺序文件、索引文件、索引顺序文件

2 文件的物理结构（文件的存储结构）

是指文件在外存上的存储组织形式，用户是看不见的，
文件的物理结构不但与存储介质的存储性能有关，而且
还与所采取的外存分配方式有关。

顺序文件、链接文件、索引文件



文件控制块

■ 文件目录

- 文件目录是一种数据结构，用于标识系统中的文件及其物理地址，将文件名映射到外存物理位置，供检索使用。

■ 目录项

- 构成文件目录的项目（目录项就是 FCB）
- 文件目录：文件控制块的有序集合。

■ 目录文件

- 为了实现对文件目录的管理，通常将文件目录以文件的形式保存在外存，这个文件就叫目录文件。

文件目录和目录文件是同一事物的两种称谓。

从用途角度来看称其为文件目录，从实现角度来看称其为目录文件。



索引结点

- 检索目录文件的过程中，只用到了文件名，仅当文件名与指定要查找的文件名相匹配时，才需从该目录项中读出该文件的物理地址，而其它信息在检索目录时不需调入内存。
- 解决方案：引入索引结点。

文件名	索引结点编号
文件名1	
文件名2	
.....	



索引结点

- 索引结点把文件名与文件描述信息分开。
- 将 FCB 分为文件名、i 结点指针 (索引 index) 和相应的 i 结点。
- 离散存放的目录结构
- 查询时只调入文件目录 (仅包括文件名和索引结点指针)，找到后才调入相应结点。



索引结点

例：在某个文件系统中，每个盘块为 512 字节，文件控制块占 64 个字节，其中文件名占 8 个字节。如果索引结点编号占 2 个字节，对一个存放在磁盘上的 256 个目录项的目录，试**比较引入索引结点前后**，为找到其中一个文件的 FCB，平均启动磁盘的次数。

解：**引入索引节点前**，每个目录项中存放的是对应文件的 FCB，故 256 个目录项的目录总共需要占用 $256 \times 64 / 512 = 32$ 个盘块。故在该目录中检索到一个文件平均启动磁盘次数为 $(1+32) / 2 = 16.5$ 。



索引结点

例：在某个文件系统中，每个盘块为 512 字节，文件控制块占 64 个字节，其中文件名占 8 个字节。如果索引结点编号占 2 个字节，对一个存放在磁盘上的 256 个目录项的目录，试**比较引入索引结点前后**，为找到其中一个文件的 FCB，平均启动磁盘的次数。

解：**引入索引节点后**，每个目录项中只需存放文件名和索引节点的编号，因此 256 个目录项的目录总共需要占用 $256 \times (8+2) / 512 = 5$ 个盘块。因此，找到匹配的目录项平均需要启动 3 次磁盘；而得到索引结点编号后还需**启动磁盘将对**
应文件的索引结点读入内存，故平均需要启动磁盘 4 次。



第 8 章

磁盘存储器的管理



外存的组织方式

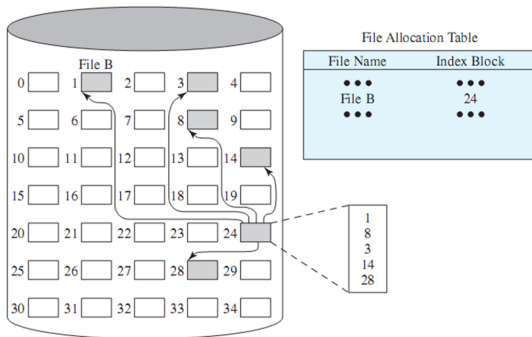
外存的组织方式

- 1 连续组织方式（顺序分配）
- 2 链接组织方式
- 3 索引组织方式



单级索引分配

- 为每个文件分配一个索引块 (表), 把分配给该文件的所有盘块号都记录在该索引块中。
- 在建立一个文件时, 只需在为之建立的目录项中填上指向该索引块的指针。



多级索引分配

- 一个大文件需要的盘块可能很多，其索引块不止一个，需要通过链指针将各索引块按序链接起来。
- 当文件太大，其索引块太多时，这种方法是低效的。
- **多级索引分配**为这些索引块再建立一级索引，即系统再分配一个（目录）索引块，作为第一级索引的索引块（两级索引）。
- 如果文件非常大时，还可用三级、四级索引分配方式。



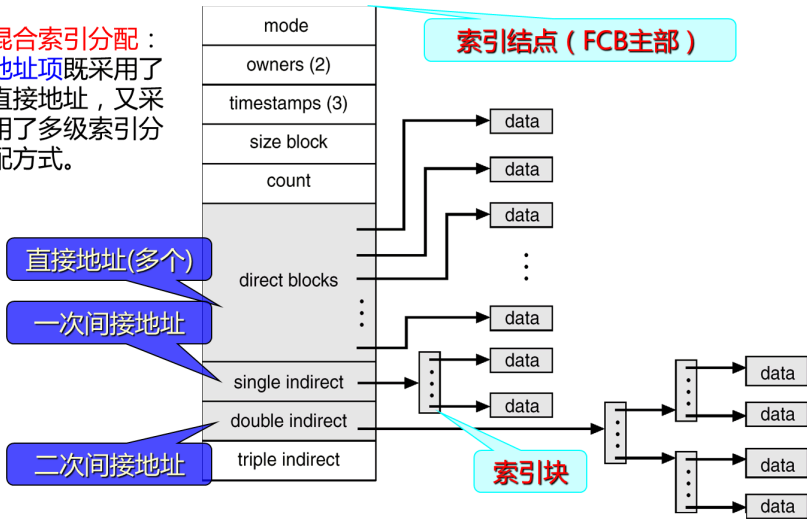
多级索引分配

- 如果每个盘块的大小为 1 KB，每个盘块号占 4 个字节，则在一个索引块中可存放 256 个盘块号。在两级索引时，最多可存放的盘块号总数 $N = 256 \times 256 = 64 \text{ K}$ 个，即所允许的文件最大长度为 64 MB。
- 如果每个盘块的大小为 4 KB，每个盘块号占 4 个字节，单级索引时所允许的文件最大长度为 $1024 \times 4 \text{ KB} = 4 \text{ MB}$
- 如果采用两级索引时最大文件长度为 $1024 \times 1024 \times 4 \text{ KB} = 4 \text{ GB}$



增量式索引组织方式 (混合索引分配)

混合索引分配：
地址项既采用了直接地址，又采用了多级索引分配方式。



练习题

例：存放在某个磁盘上的文件系统，采用混合索引分配方式，其 FCB 中共有 13 个地址项（索引项），0-9 地址项为直接地址，10 地址项为一次间接地址，11 地址项为二次间接地址，12 地址项为三次间接地址。如果每个盘块的大小为 512 字节，盘块号需要用 4 个字节来描述，而每个盘块最多存放 128 个盘块地址 $512/4=128$ 。问：

- 1 该文件系统允许文件的最大长度是多少？
- 2 将文件的字节偏移量 5000、15000、150000 转换为物理块号和块内偏移量。
- 3 假设某个文件的 FCB 已在内存，但其他信息均在外存，为了访问该文件中某个位置的内容，最少需要几次访问磁盘，最多需要几次访问磁盘？



练习题

1 该文件系统中一个文件的最大长度可达：

$10 + 128 + 128 \times 128 + 128 \times 128 \times 128 = 2113674$ 块，
共 2113674×512 字节 = 2471040 KB $\approx$ 1GB

或用如下解法：

直接索引项可管理上限： $10 \times 0.5\text{KB} = 5\text{KB}$ ；

一次间接索引项： $128 \times 0.5\text{KB} = 64\text{KB}$ ；

二次间接索引项： $128 \times 128 \times 0.5\text{KB} = 8192\text{KB}$ ；

三次间接索引项：

$128 \times 128 \times 128 \times 0.5\text{KB} = 1048576\text{KB}$ ；

文件的最大长度：

$5\text{KB} + 64\text{KB} + 8192\text{KB} + 1048576\text{KB} = 1056837\text{KB} \approx 1\text{GB}$ 。



练习题

2 5000/512 得到商为 9，余数为 392，即字节偏移量 5000 对应的逻辑块号为 9，块内偏移量为 392。可直接从该文件的 FCB 的地址项 9 处得到物理盘块号，块内偏移量为 392。

15000/512 得到商为 29，余数为 152，即字节偏移量 15000 对应的逻辑块号为 29，块内偏移量为 152。由于 $9 < 29 < 9 + 128$ ，而 $29 - 9 = 20$ ，故可从 FCB 的地址项 10，即一次间址项中得到一次间址块的地址；并从一次间址块的 20×4 的位置获得对应的物理盘块号，块内偏移量为 152。



练习题

- 2 150000/512 得到商为 292，余数为 496，即字节偏移量 150000 对应的逻辑块号为 292，块内偏移量为 496。由于 $9+128 < 292 < 9+128+128 \times 128$ ，而 $292-(9+128)=155$ ， $155/128$ 得到商为 1，余数为 27，故可从 FCB 的地址项 11，即二次间址项中得到二次间址块的地址，并从二次间址块的 1×4 的位置获得一个一次间址块的地址，再从这个一次间址块的 27×4 的位置获得对应的物理盘块号，块内偏移量为 496。



练习题

- 3 在采用混合索引分配（三级索引）的系统中，假设某个文件的FCB 已在内存，但其他信息均在外存，为了访问该文件中某个位置的内容，最少需要几次访问磁盘，最多需要几次访问磁盘？



练习题

- 3 在采用混合索引分配（三级索引）的系统中，假设某个文件的FCB 已在内存，但其他信息均在外存，为了访问该文件中某个位置的内容，最少需要几次访问磁盘，最多需要几次访问磁盘？

由于文件的 FCB 已在内存，为了访问文件中某个位置的内容，最少需要 1 次访问磁盘（即可通过直接地址直接读文件盘块），最多需要 4 次访问磁盘（第一次是读三次间址块，第二次是读二次间址块，第三次是读一次间址块，第四次是读文件盘块）。



文件存储空间的管理

- 文件存储空间的管理可采用连续分配和离散分配方式。
- 内存分配基本不采用连续分配方式。外存管理中，小文件经常采用连续分配方式，大文件采用离散分配方式。

存储空间的管理方法

- 空闲表法和空闲链法
- 位示图法
- 成组链接法
- 文件存储器空间布局



位示图法

	1	2	3	4	5	6	7	8	9	10
1	1	1	0	0	0	1	1	0	0	0
2	1	0	1	1	1	0	1	1	1	1
3	0	0	1	0	0	1	0	0	0	0
4	1	1	0	0	0	1	1	1	0	1
5	0	1	1	1	1	0	0	0	1	0
6	1	0	1	1	1	0	1	0	1	1
7	1	1	1	0	0	1	0	0	0	1



位示图法

■ 盘块的分配

- 1 顺序扫描，找一个或一组值为 0 的块。
- 2 把找到的二进制位转换成与之相应的盘块号。假定找到的“0”的二进制位在第 i 行、第 j 列，每行位数为 n ，则相应的盘块号： $b = n \times i + j$ i,j,b 都从 0 开始
- 3 修改位示图： $\text{map}[i,j]=1$ 。

■ 盘块回收

- 1 由盘块号 b 得行列号 (i,j)
 $i = b \text{ div } n$
 $j = b \text{ mod } n$ div 取整 mod 取余
- 2 修改位图： $\text{map}[i,j]=0$



练习题

例：设某系统的磁盘有 500 块，块号为：0, 1, 2, 3, ..., 499。

(1) 若用位示图法管理这 500 块的盘空间，当字长为 32 位时，此位示图占了几个字？

(2) 第 i 字的第 j 位对应的块号是多少？（其中： $i=0, 1, 2, \dots, j=0, 1, 2, 3, \dots$ ）



练习题

例：设某系统的磁盘有 500 块，块号为：0, 1, 2, 3, ..., 499。

(1) 若用位示图法管理这 500 块的盘空间，当字长为 32 位时，此位示图占了几个字？

(2) 第 i 字的第 j 位对应的块号是多少？（其中： $i=0, 1, 2, \dots, j=0, 1, 2, 3, \dots$ ）

1 位示图法就是在内存用一些字建立一张位示图，用其中的每一位表示一个盘块的使用情况，通常用“1”表示占用，“0”表示空闲。因此，本问题中位示图所占的字数为： $500 / 32 = 15.625 \approx 16$ 。

2 第 i 字的第 j 位对应的块号 $N = 32 * i + j$ 。



练习题

例：有一计算机系统利用下图所示的位示图来管理空闲盘块。每个盘块的大小为 1KB。

- (1) 现要为文件分配两个盘块，试具体说明分配过程。
- (2) 若要释放块号为 300 的盘块，应如何处理？

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1
3	1	1	1	1	1	1	0	1	1	1	1	0	1	1	1	1
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5																
6																



练习题

1 为某文件分配两个盘块的过程如下：

①顺序检索位示图，从中找到第一个值为 0 的二进制位，得到其行号 $i_1=2$ ，列号 $j_1=2$ 。第二个值为 0 的二进制位，得到其行号 $i_2=3$ ，列号 $j_2=6$ 。

②计算出找到的两个空闲块的盘块号分别为：

$$b_1 = i_1 \times 16 + j_1 = 2 \times 16 + 2 = 34$$

$$b_2 = i_2 \times 16 + j_2 = 3 \times 16 + 6 = 54$$

③修改位示图，令 $\text{map}[2,2]=\text{map}[3,6]=1$ ，并把对应的块 34 和块 54 分配出去。



练习题

2 释放磁盘的第 300 块时，应进行如下处理：

①计算出磁盘第 300 块所对应的二进制位的行号 i 和列号 j ：

$$i = 300 \div 16 = 18$$

$$j = 300 \bmod 16 = 12$$

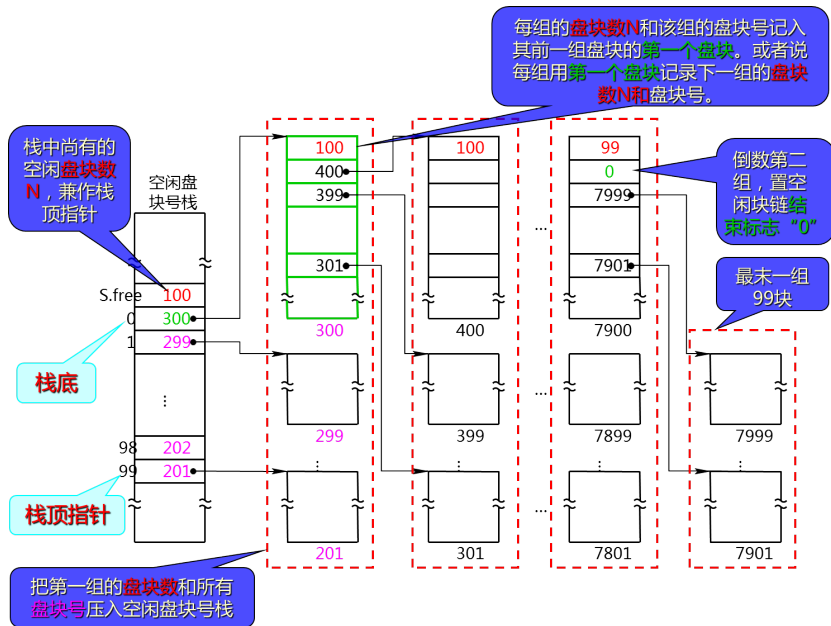
②修改位示图，令 $\text{map}[18,12]=0$ ，表示对应块为空闲块。



成组链接法

- 空闲盘块号栈：存放一组空闲盘块的盘块号（最多 100 个），以及栈中尚有的空闲盘块号数 N （兼作栈顶指针）。 $s.free(0)$ 是栈底，栈满时的栈顶为 $S.free(99)$ 。
- 所有空闲盘块被分成若干组（每组 100 个）。
- 把每组的盘块数 N 和该组的盘块号记入其前一组盘块的第一个盘块。
- 把第一组的盘块数和所有盘块号记入空闲盘块号栈。
- 最末一组置空闲块链的结束标志。





空闲盘块的分配与回收

- **分配**：从栈顶取出一空闲盘块号，将与之对应的盘块分配给用户，然后将栈顶指针下移一格。到`s.free(0)`时，所分配的磁盘块号是栈中最后一个可用盘块号，由于该块内容为下一组的所有盘块号，因此要先将该块的内容读入栈中，然后才能将该块分配出去。
- **回收**：将回收盘块的盘块号压入空闲盘块号栈的顶部，并执行**空闲盘块数 N 加 1** 操作。到栈满时，将现有栈中的 100 个盘块弹出栈，组成新的一组空闲盘块组。它们的盘块号被记入新回收的盘块中，再将其盘块号作为新栈底（记录了前一组的 100 个盘块号）。



THE END

School of Computer & Information Engineering

Henan University

Kaifeng, Henan Province

475001

China

